SKYLINE AIRLINES

Simple guide to the C project

Beginner-friendly explanations of all 34 remaining functions.

Part	What it explains
Program flow	Starting, loading data, menus, saving, and exiting.
Data structures	Flight array, booking linked list, waiting queue, and search tree.
Code guide	Every source file and all current functions.
Project files	Header, launcher, README, and saved data formats.

The admin ID is written in code/skyline.h. It is for a class demonstration and is not secure authentication.

1. Build and run

Install GCC, such as MinGW, and make sure gcc is on PATH. Double-click run.bat, or open a terminal in the project folder and run run.bat.

```
run.bat -> GCC compiles the C files -> the program starts
```

2. How the program works

• The program loads flights from flights.txt. If there are no saved flights, it creates five sample flights.

• It loads confirmed bookings and builds a flight-number search tree.

• The main menu opens the passenger portal or checks the admin ID before opening the admin portal.

• A booking uses an available seat or adds the passenger to the waiting queue.

• Cancelling returns a seat and can promote the earliest waiting passenger for that flight.

• When the program exits, it saves flights and bookings and frees allocated nodes.

3. Data structures

Structure	Simple meaning	Used for
Array	Fixed row of numbered slots.	Stores up to 50 flights.
Linked list	Nodes connected by next links.	Stores confirmed bookings.
FIFO queue	First In, First Out.	Keeps waiting passengers in order.
Binary search tree	Smaller values go left; larger values go right.	Searches flight numbers.

A pointer holds the location of another value. next, left, and right connect nodes. NULL means a node does not exist at that link.

4. Shared types and program state

code/skyline.h is the shared header. It defines data structures, constants, shared variable declarations, and function prototypes. A prototype tells the compiler a function name, inputs, and result type.

Name	Meaning
Flight	One flight: number, route, schedule, seats, and fare.
Booking	One confirmed ticket and a link to the next booking.
QueueNode / Queue	Waiting passenger nodes in order, with front and rear pointers.
TreeNode	Flight number with left links for smaller and right links for larger numbers.
flights / flightCount	The flight array and how many records it contains.
nextBookingID	ID for the next confirmed booking.
bookingHead	First node of the confirmed-booking linked list.
waitingList	Front and rear of the passenger queue.
flightRoot	Root node of the flight-number search tree.

main.c

Starts the program, loads saved data, shows the main menu, saves on exit, and frees memory.

main()

```
int main(void)
```

In simple words: Start the program, load saved data, and show the main menu. **Result:** Returns 0 when the application exits normally.

state.c

Defines the shared program data and frees the linked-list and tree nodes at exit.

freeAllMemory()

```
void freeAllMemory(void)
```

In simple words: All shared program data is defined here and declared in skyline.h. */ Flight flights[MAX_FLIGHTS]; int flightCount = 0; int nextBookingID = 1001; Booking *bookingHead = NULL; Queue waitingList = {NULL, NULL}; TreeNode *flightRoot = NULL; /* Release every dynamically allocated node before the program exits. **Result:** No value is returned.

input.c

Reads text and numbers from the keyboard and pauses between actions.

readLine()

```
void readLine(const char *prompt, char *buffer, int size)
```

In simple words: Read a whole line, including spaces, and remove its final newline. **Result:** Stores the text in the supplied character array.

readInt()

```
int readInt(const char *prompt)
```

In simple words: Read a whole line, including spaces, and remove its final newline. */ void readLine(const char *prompt, char *buffer, int size) { int extraCharacter; size_t lineLength; printf("%s", prompt); if (fgets(buffer, size, stdin) == NULL) { buffer[0] = '\0'; return; } lineLength = strlen(buffer); if (lineLength > 0 && buffer[lineLength - 1] == '\n') { buffer[lineLength - 1] = '\0'; } else { /* Discard extra characters so the next prompt starts on a new line. */ while ((extraCharacter = getchar()) != '\n' && extraCharacter != EOF) { } } } /* Keep asking until the user enters one whole number. **Result:** Returns the entered whole number, or 0 when input ends.

readFloat()

```
float readFloat(const char *prompt)
```

In simple words: Read a whole line, including spaces, and remove its final newline. */ void readLine(const char *prompt, char *buffer, int size) { int extraCharacter; size_t lineLength; printf("%s", prompt); if (fgets(buffer, size, stdin) == NULL) { buffer[0] = '\0'; return; } lineLength = strlen(buffer); if (lineLength > 0 && buffer[lineLength - 1] == '\n') { buffer[lineLength - 1] = '\0'; } else { /* Discard extra characters so the next prompt starts on a new line. */ while ((extraCharacter = getchar()) != '\n' && extraCharacter != EOF) { } } } /* Keep asking until the user enters one whole number. */ int readInt(const char *prompt) { char inputLine[100]; char extraCharacter; int value; while (1) { readLine(prompt, inputLine, sizeof(inputLine)); /* End the input loop cleanly if input is closed, such as Ctrl+Z. */ if (feof(stdin)) { return 0; } /* The extra character check rejects input such as "12abc". */ if (sscanf(inputLine, "%d %c", &value, &extraCharacter) == 1) { return value; } printf("Invalid input. Enter a whole number.\n"); } } /* Keep asking until the user enters a non-negative number. **Result:** Returns a non-negative number, or 0 when input ends.

pauseScreen()

```
void pauseScreen(void)
```

In simple words: Read a whole line, including spaces, and remove its final newline. */ void readLine(const char *prompt, char *buffer, int size) { int extraCharacter; size_t lineLength; printf("%s", prompt); if (fgets(buffer, size, stdin) == NULL) { buffer[0] = '\0'; return; } lineLength = strlen(buffer); if (lineLength > 0 && buffer[lineLength - 1] == '\n') { buffer[lineLength - 1] = '\0'; } else { /* Discard extra characters so the next prompt starts on a new line. */ while ((extraCharacter = getchar()) != '\n' && extraCharacter != EOF) { } } } /* Keep asking until the user enters one whole number. */ int readInt(const char *prompt) { char inputLine[100]; char extraCharacter; int value; while (1) { readLine(prompt, inputLine, sizeof(inputLine)); /* End the input loop cleanly if input is closed, such as Ctrl+Z. */ if (feof(stdin)) { return 0; } /* The extra character check rejects input such as "12abc". */ if (sscanf(inputLine, "%d %c", &value, &extraCharacter) == 1) { return value; } printf("Invalid input. Enter a whole number.\n"); } } /* Keep asking until the user enters a non-negative number. */ float readFloat(const char *prompt) { char inputLine[100]; char extraCharacter; float value; while (1) { readLine(prompt, inputLine, sizeof(inputLine)); if (feof(stdin)) { return 0; } if (sscanf(inputLine, "%f %c", &value, &extraCharacter) == 1 && value >= 0) { return value; } printf("Enter a valid non-negative amount.\n"); } } /* Wait for Enter before showing the next menu. **Result:** No value is returned.

flights.c

Stores and searches flights, manages the binary search tree, and adds flights.

findFlightIndex()

```
int findFlightIndex(int flightNo)
```

In simple words: Return the array position of a flight, or -1 if it does not exist. **Result:** Returns an array position, or -1 when no flight matches.

addSampleFlights()

```
void addSampleFlights(void)
```

In simple words: Return the array position of a flight, or -1 if it does not exist. */ int findFlightIndex(int flightNo) { int flightIndex; /* Flights are stored in an array, so check each item until it matches. */ for (flightIndex = 0; flightIndex < flightCount; flightIndex++) { if (flights[flightIndex].flightNo == flightNo) { return flightIndex; } } return -1; } /* Add the example flights used when no saved flight file exists. **Result:** No value is returned.

displayFlights()

```
void displayFlights(void)
```

In simple words: Return the array position of a flight, or -1 if it does not exist. */ int findFlightIndex(int flightNo) { int flightIndex; /* Flights are stored in an array, so check each item until it matches. */ for (flightIndex = 0; flightIndex < flightCount; flightIndex++) { if (flights[flightIndex].flightNo == flightNo) { return flightIndex; } } return -1; } /* Add the example flights used when no saved flight file exists. */ void addSampleFlights(void) { /* Each row lists: number, route, date, time, seats, available, fare. */ Flight sample[] = { {101, "Delhi", "Mumbai", "2026-11-10", "09:30", 20, 20, 4000}, {102, "Delhi", "Bengaluru", "2026-11-11", "12:15", 25, 25, 5200}, {103, "Mumbai", "Goa", "2026-11-12", "16:00", 15, 15, 2800}, {104, "Kolkata", "Delhi", "2026-11-13", "07:45", 18, 18, 4500}, {105, "Chennai", "Hyderabad", "2026-11-14", "18:20", 22, 22, 3100} }; int flightIndex; flightCount = (int)(sizeof(sample) / sizeof(sample[0])); for (flightIndex = 0; flightIndex < flightCount; flightIndex++) { flights[flightIndex] = sample[flightIndex]; } } /* Print every flight in the array. **Result:** No value is returned.

searchFlightByNumber()

```
void searchFlightByNumber(void)
```

In simple words: Return the array position of a flight, or -1 if it does not exist. */ int findFlightIndex(int flightNo) { int flightIndex; /* Flights are stored in an array, so check each item until it matches. */ for (flightIndex = 0; flightIndex < flightCount; flightIndex++) { if (flights[flightIndex].flightNo == flightNo) { return flightIndex; } } return -1; } /* Add the example flights used when no saved flight file exists. */ void addSampleFlights(void) { /* Each row lists: number, route, date, time, seats, available, fare. */ Flight sample[] = { {101, "Delhi", "Mumbai", "2026-11-10", "09:30", 20, 20, 4000}, {102, "Delhi", "Bengaluru", "2026-11-11", "12:15", 25, 25, 5200}, {103, "Mumbai", "Goa", "2026-11-12", "16:00", 15, 15, 2800}, {104, "Kolkata", "Delhi", "2026-11-13", "07:45", 18, 18, 4500}, {105, "Chennai", "Hyderabad", "2026-11-14", "18:20", 22, 22, 3100} }; int flightIndex; flightCount = (int)(sizeof(sample) / sizeof(sample[0])); for (flightIndex = 0; flightIndex < flightCount; flightIndex++) { flights[flightIndex] = sample[flightIndex]; } } /* Print every flight in the array. */ void displayFlights(void) { int flightIndex; if (flightCount == 0) { printf("\nNo flights available.\n"); return; } printf("\n================ AVAILABLE FLIGHTS ================\n"); printf("%-8s %-13s %-13s %-12s %-8s %-8s %-10s\n", "Flight", "From", "To", "Date", "Time", "Seats", "Fare"); printf("---" "----------------------------\n"); for (flightIndex = 0; flightIndex < flightCount; flightIndex++) { printf("%-8d %-13s %-13s %-12s %-8s %-8d Rs.%.2f\n", flights[flightIndex].flightNo, flights[flightIndex].source, flights[flightIndex].destination, flights[flightIndex].date, flights[flightIndex].time, flights[flightIndex].availableSeats, flights[flightIndex].fare); } } /* Ask for a flight number, then show its details. **Result:** No value is returned.

searchFlightsByRoute()

```
void searchFlightsByRoute(void)
```

In simple words: Return the array position of a flight, or -1 if it does not exist. */ int findFlightIndex(int flightNo) { int flightIndex; /* Flights are stored in an array, so check each item until it matches. */ for (flightIndex = 0; flightIndex < flightCount; flightIndex++) { if (flights[flightIndex].flightNo == flightNo) { return flightIndex; } } return -1; } /* Add the example flights used when no saved flight file exists. */ void addSampleFlights(void) { /* Each row lists: number, route, date, time, seats, available, fare. */ Flight sample[] = { {101, "Delhi", "Mumbai", "2026-11-10", "09:30", 20, 20, 4000}, {102, "Delhi", "Bengaluru", "2026-11-11", "12:15", 25, 25, 5200}, {103, "Mumbai", "Goa", "2026-11-12", "16:00", 15, 15, 2800}, {104, "Kolkata", "Delhi", "2026-11-13", "07:45", 18, 18, 4500}, {105, "Chennai", "Hyderabad", "2026-11-14", "18:20", 22, 22, 3100} }; int flightIndex; flightCount = (int)(sizeof(sample) / sizeof(sample[0])); for (flightIndex = 0; flightIndex < flightCount; flightIndex++) { flights[flightIndex] = sample[flightIndex]; } } /* Print every flight in the array. */ void displayFlights(void) { int flightIndex; if (flightCount == 0) { printf("\nNo flights available.\n"); return; } printf("\n================ AVAILABLE FLIGHTS ================\n"); printf("%-8s %-13s %-13s %-12s %-8s %-8s %-10s\n", "Flight", "From", "To", "Date", "Time", "Seats", "Fare"); printf("---" "----------------------------\n"); for (flightIndex = 0; flightIndex < flightCount; flightIndex++) { printf("%-8d %-13s %-13s %-12s %-8s %-8d Rs.%.2f\n", flights[flightIndex].flightNo, flights[flightIndex].source, flights[flightIndex].destination, flights[flightIndex].date, flights[flightIndex].time, flights[flightIndex].availableSeats, flights[flightIndex].fare); } } /* Ask for a flight number, then show its details. */ void searchFlightByNumber(void) { int flightNumber; int flightIndex; flightNumber = readInt("\nEnter flight number: "); flightIndex = findFlightIndex(flightNumber); if (flightIndex == -1) { printf("Flight not found.\n"); return; } printf("\nFlight Details\n"); printf("---------------------------------\n"); printf("Flight number : %d\n", flights[flightIndex].flightNo); printf("Route : %s to %s\n", flights[flightIndex].source, flights[flightIndex].destination); printf("Date : %s\n", flights[flightIndex].date); printf("Departure time : %s\n", flights[flightIndex].time); printf("Available seats : %d\n", flights[flightIndex].availableSeats); printf("Ticket fare : Rs. %.2f\n", flights[flightIndex].fare); } /* Show flights whose start and destination cities both match. **Result:** No value is returned.

insertTree()

```
TreeNode * insertTree(TreeNode *root, int flightNumber) { /* An empty link is where this flight number belongs in the BST. */ if (r
```

In simple words: Return the array position of a flight, or -1 if it does not exist. */ int findFlightIndex(int flightNo) { int flightIndex; /* Flights are stored in an array, so check each item until it matches. */ for (flightIndex = 0; flightIndex < flightCount; flightIndex++) { if (flights[flightIndex].flightNo == flightNo) { return flightIndex; } } return -1; } /* Add the example flights used when no saved flight file exists. */ void addSampleFlights(void) { /* Each row lists: number, route, date, time, seats, available, fare. */ Flight sample[] = { {101, "Delhi", "Mumbai", "2026-11-10", "09:30", 20, 20, 4000}, {102, "Delhi", "Bengaluru", "2026-11-11", "12:15", 25, 25, 5200}, {103, "Mumbai", "Goa", "2026-11-12", "16:00", 15, 15, 2800}, {104, "Kolkata", "Delhi", "2026-11-13", "07:45", 18, 18, 4500}, {105, "Chennai", "Hyderabad", "2026-11-14", "18:20", 22, 22, 3100} }; int flightIndex; flightCount = (int)(sizeof(sample) / sizeof(sample[0])); for (flightIndex = 0; flightIndex < flightCount; flightIndex++) { flights[flightIndex] = sample[flightIndex]; } } /* Print every flight in the array. */ void displayFlights(void) { int flightIndex; if (flightCount == 0) { printf("\nNo flights available.\n"); return; } printf("\n================ AVAILABLE FLIGHTS ================\n"); printf("%-8s %-13s %-13s %-12s %-8s %-8s %-10s\n", "Flight", "From", "To", "Date", "Time", "Seats", "Fare"); printf("---------- ------------- ------------- ------------ -------- -------- ----------\n"); for (flightIndex = 0; flightIndex < flightCount; flightIndex++) { printf("%-8d %-13s %-13s %-12s %-8s %-8d Rs.%.2f\n", flights[flightIndex].flightNo, flights[flightIndex].source, flights[flightIndex].destination, flights[flightIndex].date, flights[flightIndex].time, flights[flightIndex].availableSeats, flights[flightIndex].fare); } } /* Ask for a flight number, then show its details. */ void searchFlightByNumber(void) { int flightNumber; int flightIndex; flightNumber = readInt("\nEnter flight number: "); flightIndex = findFlightIndex(flightNumber); if (flightIndex == -1) { printf("Flight not found.\n"); return; } printf("\nFlight Details\n"); printf("---------------------------------\n"); printf("Flight number : %d\n", flights[flightIndex].flightNo); printf("Route : %s to %s\n", flights[flightIndex].source, flights[flightIndex].destination); printf("Date : %s\n", flights[flightIndex].date); printf("Departure time : %s\n", flights[flightIndex].time); printf("Available seats : %d\n", flights[flightIndex].availableSeats); printf("Ticket fare : Rs. %.2f\n", flights[flightIndex].fare); } /* Show flights whose start and destination cities both match. */ void searchFlightsByRoute(void) { char source[CITY_LEN]; char destination[CITY_LEN]; int flightIndex; int found = 0; readLine("\nDeparture city: ", source, sizeof(source)); readLine("Destination city: ", destination, sizeof(destination)); for (flightIndex = 0; flightIndex < flightCount; flightIndex++) { if (strcmp(flights[flightIndex].source, source) == 0 && strcmp(flights[flightIndex].destination, destination) == 0) { printf("\nFlight %d | %s to %s\n", flights[flightIndex].flightNo, flights[flightIndex].source, flights[flightIndex].destination); printf("Date: %s | Time: %s\n", flights[flightIndex].date, flights[flightIndex].time); printf("Seats: %d | Fare: Rs. %.2f\n", flights[flightIndex].availableSeats, flights[flightIndex].fare); found = 1; } } if (!found) { printf("No matching route found.\n"); printf("City names are case-sensitive.\n"); } } /* Insert a flight number into its correct place in the BST. **Result:** Returns the updated root of the tree or subtree.

searchTree()

```
TreeNode * searchTree(TreeNode *root, int flightNumber) { /* Each comparison chooses one branch, avoiding a full array scan. */ if
```

In simple words: Return the array position of a flight, or -1 if it does not exist. */ int findFlightIndex(int flightNo) { int flightIndex; /* Flights are stored in an array, so check each item until it matches. */ for (flightIndex = 0; flightIndex < flightCount; flightIndex++) { if (flights[flightIndex].flightNo == flightNo) { return flightIndex; } } return -1; } /* Add the example flights used when no saved flight file exists. */ void addSampleFlights(void) { /* Each row lists: number, route, date, time, seats, available, fare. */ Flight sample[] = { {101, "Delhi", "Mumbai", "2026-11-10", "09:30", 20, 20, 4000}, {102, "Delhi", "Bengaluru", "2026-11-11", "12:15", 25, 25, 5200}, {103, "Mumbai", "Goa", "2026-11-12", "16:00", 15, 15, 2800}, {104, "Kolkata", "Delhi", "2026-11-13", "07:45", 18, 18, 4500}, {105, "Chennai", "Hyderabad", "2026-11-14", "18:20", 22, 22, 3100} }; int flightIndex; flightCount = (int)(sizeof(sample) / sizeof(sample[0])); for (flightIndex = 0; flightIndex < flightCount; flightIndex++) { flights[flightIndex] = sample[flightIndex]; } } /* Print every flight in the array. */ void displayFlights(void) { int flightIndex; if (flightCount == 0) { printf("\nNo flights available.\n"); return; } printf("\n================ AVAILABLE FLIGHTS ================\n"); printf("%-8s %-13s %-13s %-12s %-8s %-8s %-10s\n", "Flight", "From", "To", "Date", "Time", "Seats", "Fare"); printf("---------- ------------- ------------- ------------ -------- -------- ----------\n"); for (flightIndex = 0; flightIndex < flightCount; flightIndex++) { printf("%-8d %-13s %-13s %-12s %-8s %-8d Rs.%.2f\n", flights[flightIndex].flightNo, flights[flightIndex].source, flights[flightIndex].destination, flights[flightIndex].date, flights[flightIndex].time, flights[flightIndex].availableSeats, flights[flightIndex].fare); } } /* Ask for a flight number, then show its details. */ void searchFlightByNumber(void) { int flightNumber; int flightIndex; flightNumber = readInt("\nEnter flight number: "); flightIndex = findFlightIndex(flightNumber); if (flightIndex == -1) { printf("Flight not found.\n"); return; } printf("\nFlight Details\n"); printf("---------------------------------\n"); printf("Flight number : %d\n", flights[flightIndex].flightNo); printf("Route : %s to %s\n", flights[flightIndex].source, flights[flightIndex].destination); printf("Date : %s\n", flights[flightIndex].date); printf("Departure time : %s\n", flights[flightIndex].time); printf("Available seats : %d\n", flights[flightIndex].availableSeats); printf("Ticket fare : Rs. %.2f\n", flights[flightIndex].fare); } /* Show flights whose start and destination cities both match. */ void searchFlightsByRoute(void) { char source[CITY_LEN]; char destination[CITY_LEN]; int flightIndex; int found = 0; readLine("\nDeparture city: ", source, sizeof(source)); readLine("Destination city: ", destination, sizeof(destination)); for (flightIndex = 0; flightIndex < flightCount; flightIndex++) { if (strcmp(flights[flightIndex].source, source) == 0 && strcmp(flights[flightIndex].destination, destination) == 0) { printf("\nFlight %d | %s to %s\n", flights[flightIndex].flightNo, flights[flightIndex].source, flights[flightIndex].destination); printf("Date: %s | Time: %s\n", flights[flightIndex].date, flights[flightIndex].time); printf("Seats: %d | Fare: Rs. %.2f\n", flights[flightIndex].availableSeats, flights[flightIndex].fare); found = 1; } } if (!found) { printf("No matching route found.\n"); printf("City names are case-sensitive.\n"); } } /* Insert a flight number into its correct place in the BST. */ TreeNode *insertTree(TreeNode *root, int flightNumber) { /* An empty link is where this flight number belongs in the BST. */ if (root == NULL) { TreeNode *newNode = (TreeNode *)malloc(sizeof(TreeNode)); if (newNode == NULL) { return NULL; } newNode->flightNo = flightNumber; newNode->left = NULL; newNode->right = NULL; return newNode; } /* Smaller flight numbers go left; larger numbers go right. */ if (flightNumber < root->flightNo) { root->left = insertTree(root->left, flightNumber); } else if (flightNumber > root->flightNo) { root->right = insertTree(root->right, flightNumber); } return root; } /* Find a flight number by following the BST's left or right links. **Result:** Returns the matching node, or NULL if it is not found.

freeTree()

```
void freeTree(TreeNode *root) { if (root == NULL)
```

In simple words: Return the array position of a flight, or -1 if it does not exist. */ int findFlightIndex(int flightNo) { int flightIndex; /* Flights are stored in an array, so check each item until it matches. */ for (flightIndex = 0; flightIndex < flightCount; flightIndex++) { if (flights[flightIndex].flightNo == flightNo) { return flightIndex; } } return -1; } /* Add the example flights used when no saved flight file exists. */ void addSampleFlights(void) { /* Each row lists: number, route, date, time, seats, available, fare. */ Flight sample[] = { {101, "Delhi", "Mumbai", "2026-11-10", "09:30", 20, 20, 4000}, {102, "Delhi", "Bengaluru", "2026-11-11", "12:15", 25, 25, 5200}, {103, "Mumbai", "Goa", "2026-11-12", "16:00", 15, 15, 2800}, {104, "Kolkata", "Delhi", "2026-11-13", "07:45", 18, 18, 4500}, {105, "Chennai", "Hyderabad", "2026-11-14", "18:20", 22, 22, 3100} }; int flightIndex; flightCount = (int)(sizeof(sample) / sizeof(sample[0])); for (flightIndex = 0; flightIndex < flightCount; flightIndex++) { flights[flightIndex] = sample[flightIndex]; } } /* Print every flight in the array. */ void displayFlights(void) { int flightIndex; if (flightCount == 0) { printf("\nNo flights available.\n"); return; } printf("\n================ AVAILABLE FLIGHTS ================\n"); printf("%-8s %-13s %-13s %-12s %-8s %-8s %-10s\n", "Flight", "From", "To", "Date", "Time", "Seats", "Fare"); printf("---" "----------------------------\n"); for (flightIndex = 0; flightIndex < flightCount; flightIndex++) { printf("%-8d %-13s %-13s %-12s %-8s %-8d Rs.%.2f\n", flights[flightIndex].flightNo, flights[flightIndex].source, flights[flightIndex].destination, flights[flightIndex].date, flights[flightIndex].time, flights[flightIndex].availableSeats, flights[flightIndex].fare); } } /* Ask for a flight number, then show its details. */ void searchFlightByNumber(void) { int flightNumber; int flightIndex; flightNumber = readInt("\nEnter flight number: "); flightIndex = findFlightIndex(flightNumber); if (flightIndex == -1) { printf("Flight not found.\n"); return; } printf("\nFlight Details\n"); printf("---------------------------------\n"); printf("Flight number : %d\n", flights[flightIndex].flightNo); printf("Route : %s to %s\n", flights[flightIndex].source, flights[flightIndex].destination); printf("Date : %s\n", flights[flightIndex].date); printf("Departure time : %s\n", flights[flightIndex].time); printf("Available seats : %d\n", flights[flightIndex].availableSeats); printf("Ticket fare : Rs. %.2f\n", flights[flightIndex].fare); } /* Show flights whose start and destination cities both match. */ void searchFlightsByRoute(void) { char source[CITY_LEN]; char destination[CITY_LEN]; int flightIndex; int found = 0; readLine("\nDeparture city: ", source, sizeof(source)); readLine("Destination city: ", destination, sizeof(destination)); for (flightIndex = 0; flightIndex < flightCount; flightIndex++) { if (strcmp(flights[flightIndex].source, source) == 0 && strcmp(flights[flightIndex].destination, destination) == 0) { printf("\nFlight %d | %s to %s\n", flights[flightIndex].flightNo, flights[flightIndex].source, flights[flightIndex].destination); printf("Date: %s | Time: %s\n", flights[flightIndex].date, flights[flightIndex].time); printf("Seats: %d | Fare: Rs. %.2f\n", flights[flightIndex].availableSeats, flights[flightIndex].fare); found = 1; } } if (!found) { printf("No matching route found.\n"); printf("City names are case-sensitive.\n"); } } /* Insert a flight number into its correct place in the BST. */ TreeNode *insertTree(TreeNode *root, int flightNumber) { /* An empty link is where this flight number belongs in the BST. */ if (root == NULL) { TreeNode *newNode = (TreeNode *)malloc(sizeof(TreeNode)); if (newNode == NULL) { return NULL; } newNode->flightNo = flightNumber; newNode->left = NULL; newNode->right = NULL; return newNode; } /* Smaller flight numbers go left; larger numbers go right. */ if (flightNumber < root->flightNo) { root->left = insertTree(root->left, flightNumber); } else if (flightNumber > root->flightNo) { root->right = insertTree(root->right, flightNumber); } return root; } /* Find a flight number by following the BST's left or right links. */ TreeNode *searchTree(TreeNode *root, int flightNumber) { /* Each comparison chooses one branch, avoiding a full array scan. */ if (root == NULL || root->flightNo == flightNumber) { return root; } if (flightNumber < root->flightNo) { return searchTree(root->left, flightNumber); } return searchTree(root->right, flightNumber); } /* Free this tree's children before freeing the parent node. **Result:** No value is returned.

rebuildFlightTree()

```
void rebuildFlightTree(void)
```

In simple words: Return the array position of a flight, or -1 if it does not exist. */ int findFlightIndex(int flightNo) { int flightIndex; /* Flights are stored in an array, so check each item until it matches. */ for (flightIndex = 0; flightIndex < flightCount; flightIndex++) { if (flights[flightIndex].flightNo == flightNo) { return flightIndex; } } return -1; } /* Add the example flights used when no saved flight file exists. */ void addSampleFlights(void) { /* Each row lists: number, route, date, time, seats, available, fare. */ Flight sample[] = { {101, "Delhi", "Mumbai", "2026-11-10", "09:30", 20, 20, 4000}, {102, "Delhi", "Bengaluru", "2026-11-11", "12:15", 25, 25, 5200}, {103, "Mumbai", "Goa", "2026-11-12", "16:00", 15, 15, 2800}, {104, "Kolkata", "Delhi", "2026-11-13", "07:45", 18, 18, 4500}, {105, "Chennai", "Hyderabad", "2026-11-14", "18:20", 22, 22, 3100} }; int flightIndex; flightCount = (int)(sizeof(sample) / sizeof(sample[0])); for (flightIndex = 0; flightIndex < flightCount; flightIndex++) { flights[flightIndex] = sample[flightIndex]; } } /* Print every flight in the array. */ void displayFlights(void) { int flightIndex; if (flightCount == 0) { printf("\nNo flights available.\n"); return; } printf("\n================ AVAILABLE FLIGHTS ================\n"); printf("%-8s %-13s %-13s %-12s %-8s %-8s %-10s\n", "Flight", "From", "To", "Date", "Time", "Seats", "Fare"); printf("---" "----------------------------\n"); for (flightIndex = 0; flightIndex < flightCount; flightIndex++) { printf("%-8d %-13s %-13s %-12s %-8s %-8d Rs.%.2f\n", flights[flightIndex].flightNo, flights[flightIndex].source, flights[flightIndex].destination, flights[flightIndex].date, flights[flightIndex].time, flights[flightIndex].availableSeats, flights[flightIndex].fare); } } /* Ask for a flight number, then show its details. */ void searchFlightByNumber(void) { int flightNumber; int flightIndex; flightNumber = readInt("\nEnter flight number: "); flightIndex = findFlightIndex(flightNumber); if (flightIndex == -1) { printf("Flight not found.\n"); return; } printf("\nFlight Details\n"); printf("---------------------------------\n"); printf("Flight number : %d\n", flights[flightIndex].flightNo); printf("Route : %s to %s\n", flights[flightIndex].source, flights[flightIndex].destination); printf("Date : %s\n", flights[flightIndex].date); printf("Departure time : %s\n", flights[flightIndex].time); printf("Available seats : %d\n", flights[flightIndex].availableSeats); printf("Ticket fare : Rs. %.2f\n", flights[flightIndex].fare); } /* Show flights whose start and destination cities both match. */ void searchFlightsByRoute(void) { char source[CITY_LEN]; char destination[CITY_LEN]; int flightIndex; int found = 0; readLine("\nDeparture city: ", source, sizeof(source)); readLine("Destination city: ", destination, sizeof(destination)); for (flightIndex = 0; flightIndex < flightCount; flightIndex++) { if (strcmp(flights[flightIndex].source, source) == 0 && strcmp(flights[flightIndex].destination, destination) == 0) { printf("\nFlight %d | %s to %s\n", flights[flightIndex].flightNo, flights[flightIndex].source, flights[flightIndex].destination); printf("Date: %s | Time: %s\n", flights[flightIndex].date, flights[flightIndex].time); printf("Seats: %d | Fare: Rs. %.2f\n", flights[flightIndex].availableSeats, flights[flightIndex].fare); found = 1; } } if (!found) { printf("No matching route found.\n"); printf("City names are case-sensitive.\n"); } } /* Insert a flight number into its correct place in the BST. */ TreeNode *insertTree(TreeNode *root, int flightNumber) { /* An empty link is where this flight number belongs in the BST. */ if (root == NULL) { TreeNode *newNode = (TreeNode *)malloc(sizeof(TreeNode)); if (newNode == NULL) { return NULL; } newNode->flightNo = flightNumber; newNode->left = NULL; newNode->right = NULL; return newNode; } /* Smaller flight numbers go left; larger numbers go right. */ if (flightNumber < root->flightNo) { root->left = insertTree(root->left, flightNumber); } else if (flightNumber > root->flightNo) { root->right = insertTree(root->right, flightNumber); } return root; } /* Find a flight number by following the BST's left or right links. */ TreeNode *searchTree(TreeNode *root, int flightNumber) { /* Each comparison chooses one branch, avoiding a full array scan. */ if (root == NULL || root->flightNo == flightNumber) { return root; } if (flightNumber < root->flightNo) { return searchTree(root->left, flightNumber); } return searchTree(root->right, flightNumber); } /* Free this tree's children before freeing the parent node. */ void freeTree(TreeNode *root) { if (root == NULL) { return; } freeTree(root->left); freeTree(root->right); free(root); } /* Recreate the BST from the flight array. **Result:** No value is returned.

searchFlightBST()

```
void searchFlightBST(void)
```

In simple words: Return the array position of a flight, or -1 if it does not exist. */ int findFlightIndex(int flightNo) { int flightIndex; /* Flights are stored in an array, so check each item until it matches. */ for (flightIndex = 0; flightIndex < flightCount; flightIndex++) { if (flights[flightIndex].flightNo == flightNo) { return flightIndex; } } return -1; } /* Add the example flights used when no saved flight file exists. */ void addSampleFlights(void) { /* Each row lists: number, route, date, time, seats, available, fare. */ Flight sample[] = { {101, "Delhi", "Mumbai", "2026-11-10", "09:30", 20, 20, 4000}, {102, "Delhi", "Bengaluru", "2026-11-11", "12:15", 25, 25, 5200}, {103, "Mumbai", "Goa", "2026-11-12", "16:00", 15, 15, 2800}, {104, "Kolkata", "Delhi", "2026-11-13", "07:45", 18, 18, 4500}, {105, "Chennai", "Hyderabad", "2026-11-14", "18:20", 22, 22, 3100} }; int flightIndex; flightCount = (int)(sizeof(sample) / sizeof(sample[0])); for (flightIndex = 0; flightIndex < flightCount; flightIndex++) { flights[flightIndex] = sample[flightIndex]; } } /* Print every flight in the array. */ void displayFlights(void) { int flightIndex; if (flightCount == 0) { printf("\nNo flights available.\n"); return; } printf("\n================ AVAILABLE FLIGHTS ================\n"); printf("%-8s %-13s %-13s %-12s %-8s %-8s %-10s\n", "Flight", "From", "To", "Date", "Time", "Seats", "Fare"); printf("---" "----------------------------\n"); for (flightIndex = 0; flightIndex < flightCount; flightIndex++) { printf("%-8d %-13s %-13s %-12s %-8s %-8d Rs.%.2f\n", flights[flightIndex].flightNo, flights[flightIndex].source, flights[flightIndex].destination, flights[flightIndex].date, flights[flightIndex].time, flights[flightIndex].availableSeats, flights[flightIndex].fare); } } /* Ask for a flight number, then show its details. */ void searchFlightByNumber(void) { int flightNumber; int flightIndex; flightNumber = readInt("\nEnter flight number: "); flightIndex = findFlightIndex(flightNumber); if (flightIndex == -1) { printf("Flight not found.\n"); return; } printf("\nFlight Details\n"); printf("----------------------------------\n"); printf("Flight number : %d\n", flights[flightIndex].flightNo); printf("Route : %s to %s\n", flights[flightIndex].source, flights[flightIndex].destination); printf("Date : %s\n", flights[flightIndex].date); printf("Departure time : %s\n", flights[flightIndex].time); printf("Available seats : %d\n", flights[flightIndex].availableSeats); printf("Ticket fare : Rs. %.2f\n", flights[flightIndex].fare); } /* Show flights whose start and destination cities both match. */ void searchFlightsByRoute(void) { char source[CITY_LEN]; char destination[CITY_LEN]; int flightIndex; int found = 0; readLine("\nDeparture city: ", source, sizeof(source)); readLine("Destination city: ", destination, sizeof(destination)); for (flightIndex = 0; flightIndex < flightCount; flightIndex++) { if (strcmp(flights[flightIndex].source, source) == 0 && strcmp(flights[flightIndex].destination, destination) == 0) { printf("\nFlight %d | %s to %s\n", flights[flightIndex].flightNo, flights[flightIndex].source, flights[flightIndex].destination); printf("Date: %s | Time: %s\n", flights[flightIndex].date, flights[flightIndex].time); printf("Seats: %d | Fare: Rs. %.2f\n", flights[flightIndex].availableSeats, flights[flightIndex].fare); found = 1; } } if (!found) { printf("No matching route found.\n"); printf("City names are case-sensitive.\n"); } } /* Insert a flight number into its correct place in the BST. */ TreeNode *insertTree(TreeNode *root, int flightNumber) { /* An empty link is where this flight number belongs in the BST. */ if (root == NULL) { TreeNode *newNode = (TreeNode *)malloc(sizeof(TreeNode)); if (newNode == NULL) { return NULL; } newNode->flightNo = flightNumber; newNode->left = NULL; newNode->right = NULL; return newNode; } /* Smaller flight numbers go left; larger numbers go right. */ if (flightNumber < root->flightNo) { root->left = insertTree(root->left, flightNumber); } else if (flightNumber > root->flightNo) { root->right = insertTree(root->right, flightNumber); } return root; } /* Find a flight number by following the BST's left or right links. */ TreeNode *searchTree(TreeNode *root, int flightNumber) { /* Each comparison chooses one branch, avoiding a full array scan. */ if (root == NULL || root->flightNo == flightNumber) { return root; } if (flightNumber < root->flightNo) { return searchTree(root->left, flightNumber); } return searchTree(root->right, flightNumber); } /* Free this tree's children before freeing the parent node. */ void freeTree(TreeNode *root) { if (root == NULL) { return; } freeTree(root->left); freeTree(root->right); free(root); } /* Recreate the BST from the flight array. */ void rebuildFlightTree(void) { int flightIndex; /* Rebuild after flight changes so the tree matches the flight array. */ freeTree(flightRoot); flightRoot = NULL; for (flightIndex = 0; flightIndex < flightCount; flightIndex++) { flightRoot = insertTree(flightRoot, flights[flightIndex].flightNo); } } /* Search the BST, then print details from the flight array. **Result:** No value is returned.

addFlight()

```
void addFlight(void)
```

In simple words: Return the array position of a flight, or -1 if it does not exist. */ int findFlightIndex(int flightNo) { int flightIndex; /* Flights are stored in an array, so check each item until it matches. */ for (flightIndex = 0; flightIndex < flightCount; flightIndex++) { if (flights[flightIndex].flightNo == flightNo) { return flightIndex; } } return -1; } /* Add the example flights used when no saved flight file exists. */ void addSampleFlights(void) { /* Each row lists: number, route, date, time, seats, available, fare. */ Flight sample[] = { {101, "Delhi", "Mumbai", "2026-11-10", "09:30", 20, 20, 4000}, {102, "Delhi", "Bengaluru", "2026-11-11", "12:15", 25, 25, 5200}, {103, "Mumbai", "Goa", "2026-11-12", "16:00", 15, 15, 2800}, {104, "Kolkata", "Delhi", "2026-11-13", "07:45", 18, 18, 4500}, {105, "Chennai", "Hyderabad", "2026-11-14", "18:20", 22, 22, 3100} }; int flightIndex; flightCount = (int)(sizeof(sample) / sizeof(sample[0])); for (flightIndex = 0; flightIndex < flightCount; flightIndex++) { flights[flightIndex] = sample[flightIndex]; } } /* Print every flight in the array. */ void displayFlights(void) { int flightIndex; if (flightCount == 0) { printf("\nNo flights available.\n"); return; } printf("\n================ AVAILABLE FLIGHTS ================\n"); printf("%-8s %-13s %-13s %-12s %-8s %-8s %-10s\n", "Flight", "From", "To", "Date", "Time", "Seats", "Fare"); printf("---" "----------------------------\n"); for (flightIndex = 0; flightIndex < flightCount; flightIndex++) { printf("%-8d %-13s %-13s %-12s %-8s %-8d Rs.%.2f\n", flights[flightIndex].flightNo, flights[flightIndex].source, flights[flightIndex].destination, flights[flightIndex].date, flights[flightIndex].time, flights[flightIndex].availableSeats, flights[flightIndex].fare); } } /* Ask for a flight number, then show its details. */ void searchFlightByNumber(void) { int flightNumber; int flightIndex; flightNumber = readInt("\nEnter flight number: "); flightIndex = findFlightIndex(flightNumber); if (flightIndex == -1) { printf("Flight not found.\n"); return; } printf("\nFlight Details\n"); printf("---------------------------------\n"); printf("Flight number : %d\n", flights[flightIndex].flightNo); printf("Route : %s to %s\n", flights[flightIndex].source, flights[flightIndex].destination); printf("Date : %s\n", flights[flightIndex].date); printf("Departure time : %s\n", flights[flightIndex].time); printf("Available seats : %d\n", flights[flightIndex].availableSeats); printf("Ticket fare : Rs. %.2f\n", flights[flightIndex].fare); } /* Show flights whose start and destination cities both match. */ void searchFlightsByRoute(void) { char source[CITY_LEN]; char destination[CITY_LEN]; int flightIndex; int found = 0; readLine("\nDeparture city: ", source, sizeof(source)); readLine("Destination city: ", destination, sizeof(destination)); for (flightIndex = 0; flightIndex < flightCount; flightIndex++) { if (strcmp(flights[flightIndex].source, source) == 0 && strcmp(flights[flightIndex].destination, destination) == 0) { printf("\nFlight %d | %s to %s\n", flights[flightIndex].flightNo, flights[flightIndex].source, flights[flightIndex].destination); printf("Date: %s | Time: %s\n", flights[flightIndex].date, flights[flightIndex].time); printf("Seats: %d | Fare: Rs. %.2f\n", flights[flightIndex].availableSeats, flights[flightIndex].fare); found = 1; } } if (!found) { printf("No matching route found.\n"); printf("City names are case-sensitive.\n"); } } /* Insert a flight number into its correct place in the BST. */ TreeNode *insertTree(TreeNode *root, int flightNumber) { /* An empty link is where this flight number belongs in the BST. */ if (root == NULL) { TreeNode *newNode = (TreeNode *)malloc(sizeof(TreeNode)); if (newNode == NULL) { return NULL; } newNode->flightNo = flightNumber; newNode->left = NULL; newNode->right = NULL; return newNode; } /* Smaller flight numbers go left; larger numbers go right. */ if (flightNumber < root->flightNo) { root->left = insertTree(root->left, flightNumber); } else if (flightNumber > root->flightNo) { root->right = insertTree(root->right, flightNumber); } return root; } /* Find a flight number by following the BST's left or right links. */ TreeNode *searchTree(TreeNode *root, int flightNumber) { /* Each comparison chooses one branch, avoiding a full array scan. */ if (root == NULL || root->flightNo == flightNumber) { return root; } if (flightNumber < root->flightNo) { return searchTree(root->left, flightNumber); } return searchTree(root->right, flightNumber); } /* Free this tree's children before freeing the parent node. */ void freeTree(TreeNode *root) { if (root == NULL) { return; } freeTree(root->left); freeTree(root->right); free(root); } /* Recreate the BST from the flight array. */ void rebuildFlightTree(void) { int flightIndex; /* Rebuild after flight changes so the tree matches the flight array. */ freeTree(flightRoot); flightRoot = NULL; for (flightIndex = 0; flightIndex < flightCount; flightIndex++) { flightRoot = insertTree(flightRoot, flights[flightIndex].flightNo); } } /* Search the BST, then print details from the flight array. */ void searchFlightBST(void) { int flightNumber; int flightIndex; TreeNode *matchingNode; flightNumber = readInt("\nEnter flight number for BST search: "); matchingNode = searchTree(flightRoot, flightNumber); if (matchingNode == NULL) { printf("Flight not found in the BST.\n"); return; } flightIndex = findFlightIndex(flightNumber); if (flightIndex != -1) { printf("Flight found using BST!\n"); printf("%d: %s to %s | Fare: Rs. %.2f\n", flights[flightIndex].flightNo, flights[flightIndex].source, flights[flightIndex].destination, flights[flightIndex].fare); } } /* Ask for flight details and add a new flight to the array and BST. **Result:** No value is returned.

bookings.c

Manages confirmed bookings, the waiting queue, bookings, and cancellations.

findBooking()

```
Booking * findBooking(int bookingID)
```

In simple words: Find and return a booking by its ID. **Result:** Returns the result shown by its return type.

appendBooking()

```
int appendBooking(int bookingID, const char *name, int age, int flightNo)
```

In simple words: Find and return a booking by its ID. */ Booking *findBooking(int bookingID) { Booking *currentBooking = bookingHead; while (currentBooking != NULL) { if (currentBooking->bookingID == bookingID) { return currentBooking; } currentBooking = currentBooking->next; } return NULL; } /* Make a booking node and attach it to the end of the booking list. **Result:** Returns 1 on success, or 0 if memory allocation fails.

displayAllBookings()

```
void displayAllBookings(void)
```

In simple words: Find and return a booking by its ID. */ Booking *findBooking(int bookingID) { Booking *currentBooking = bookingHead; while (currentBooking != NULL) { if (currentBooking->bookingID == bookingID) { return currentBooking; } currentBooking = currentBooking->next; } return NULL; } /* Make a booking node and attach it to the end of the booking list. */ int appendBooking(int bookingID, const char *name, int age, int flightNo) { Booking *newBooking; Booking *lastBooking; newBooking = (Booking *)malloc(sizeof(Booking)); if (newBooking == NULL) { printf("Memory allocation failed.\n"); return 0; } newBooking->bookingID = bookingID; newBooking->age = age; newBooking->flightNo = flightNo; snprintf(newBooking->passengerName, sizeof(newBooking->passengerName), "%s", name); newBooking->next = NULL; /* Add the new booking at the end of the singly linked list. */ if (bookingHead == NULL) { bookingHead = newBooking; } else { lastBooking = bookingHead; while (lastBooking->next != NULL) { lastBooking = lastBooking->next; } lastBooking->next = newBooking; } return 1; } /* Print each booking in the linked list. **Result:** No value is returned.

viewBooking()

```
void viewBooking(int passengerMode)
```

In simple words: Find and return a booking by its ID. */ Booking *findBooking(int bookingID) { Booking *currentBooking = bookingHead; while (currentBooking != NULL) { if (currentBooking->bookingID == bookingID) { return currentBooking; } currentBooking = currentBooking->next; } return NULL; } /* Make a booking node and attach it to the end of the booking list. */ int appendBooking(int bookingID, const char *name, int age, int flightNo) { Booking *newBooking; Booking *lastBooking; newBooking = (Booking *)malloc(sizeof(Booking)); if (newBooking == NULL) { printf("Memory allocation failed.\n"); return 0; } newBooking->bookingID = bookingID; newBooking->age = age; newBooking->flightNo = flightNo; snprintf(newBooking->passengerName, sizeof(newBooking->passengerName), "%s", name); newBooking->next = NULL; /* Add the new booking at the end of the singly linked list. */ if (bookingHead == NULL) { bookingHead = newBooking; } else { lastBooking = bookingHead; while (lastBooking->next != NULL) { lastBooking = lastBooking->next; } lastBooking->next = newBooking; } return 1; } /* Print each booking in the linked list. */ void displayAllBookings(void) { Booking *currentBooking = bookingHead; if (currentBooking == NULL) { printf("\nNo confirmed bookings.\n"); return; } printf("\n========== CONFIRMED BOOKINGS ==========\n"); while (currentBooking != NULL) { printf("Booking ID: %d | Passenger: %s\n", currentBooking->bookingID, currentBooking->passengerName); printf("Age: %d | Flight: %d\n\n", currentBooking->age, currentBooking->flightNo); currentBooking = currentBooking->next; } } /* Show a booking; passengers must also enter the booking name. **Result:** No value is returned.

enqueueWaiting()

```
void enqueueWaiting(const char *name, int age, int flightNo)
```

In simple words: Find and return a booking by its ID. */ Booking *findBooking(int bookingID) { Booking *currentBooking = bookingHead; while (currentBooking != NULL) { if (currentBooking->bookingID == bookingID) { return currentBooking; } currentBooking = currentBooking->next; } return NULL; } /* Make a booking node and attach it to the end of the booking list. */ int appendBooking(int bookingID, const char *name, int age, int flightNo) { Booking *newBooking; Booking *lastBooking; newBooking = (Booking *)malloc(sizeof(Booking)); if (newBooking == NULL) { printf("Memory allocation failed.\n"); return 0; } newBooking->bookingID = bookingID; newBooking->age = age; newBooking->flightNo = flightNo; snprintf(newBooking->passengerName, sizeof(newBooking->passengerName), "%s", name); newBooking->next = NULL; /* Add the new booking at the end of the singly linked list. */ if (bookingHead == NULL) { bookingHead = newBooking; } else { lastBooking = bookingHead; while (lastBooking->next != NULL) { lastBooking = lastBooking->next; } lastBooking->next = newBooking; } return 1; } /* Print each booking in the linked list. */ void displayAllBookings(void) { Booking *currentBooking = bookingHead; if (currentBooking == NULL) { printf("\nNo confirmed bookings.\n"); return; } printf("\n========== CONFIRMED BOOKINGS ==========\n"); while (currentBooking != NULL) { printf("Booking ID: %d | Passenger: %s\n", currentBooking->bookingID, currentBooking->passengerName); printf("Age: %d | Flight: %d\n\n", currentBooking->age, currentBooking->flightNo); currentBooking = currentBooking->next; } } /* Show a booking; passengers must also enter the booking name. */ void viewBooking(int passengerMode) { int bookingID; int flightIndex; char passengerName[NAME_LEN]; Booking *booking; bookingID = readInt("\nEnter booking ID: "); booking = findBooking(bookingID); if (booking == NULL) { printf("Booking not found.\n"); return; } if (passengerMode) { readLine("Enter passenger name used for booking: ", passengerName, sizeof(passengerName)); if (strcmp(passengerName, booking->passengerName) != 0) { printf("Passenger name does not match.\n"); return; } } flightIndex = findFlightIndex(booking->flightNo); printf("\n========== SKYLINE AIRLINES ==========\n"); printf(" E-TICKET\n"); printf("--------------------------------------\n"); printf("Booking ID : %d\n", booking->bookingID); printf("Passenger : %s\n", booking->passengerName); printf("Age : %d\n", booking->age); printf("Flight : %d\n", booking->flightNo); if (flightIndex != -1) { printf("Route : %s to %s\n", flights[flightIndex].source, flights[flightIndex].destination); printf("Date : %s\n", flights[flightIndex].date); printf("Time : %s\n", flights[flightIndex].time); printf("Fare : Rs. %.2f\n", flights[flightIndex].fare); } printf("======================================\n"); } /* Add a passenger to the back of the waiting queue. **Result:** No value is returned.

displayWaitingList()

```
void displayWaitingList(void)
```

In simple words: Find and return a booking by its ID. */ Booking *findBooking(int bookingID) { Booking *currentBooking = bookingHead; while (currentBooking != NULL) { if (currentBooking->bookingID == bookingID) { return currentBooking; } currentBooking = currentBooking->next; } return NULL; } /* Make a booking node and attach it to the end of the booking list. */ int appendBooking(int bookingID, const char *name, int age, int flightNo) { Booking *newBooking; Booking *lastBooking; newBooking = (Booking *)malloc(sizeof(Booking)); if (newBooking == NULL) { printf("Memory allocation failed.\n"); return 0; } newBooking->bookingID = bookingID; newBooking->age = age; newBooking->flightNo = flightNo; snprintf(newBooking->passengerName, sizeof(newBooking->passengerName), "%s", name); newBooking->next = NULL; /* Add the new booking at the end of the singly linked list. */ if (bookingHead == NULL) { bookingHead = newBooking; } else { lastBooking = bookingHead; while (lastBooking->next != NULL) { lastBooking = lastBooking->next; } lastBooking->next = newBooking; } return 1; } /* Print each booking in the linked list. */ void displayAllBookings(void) { Booking *currentBooking = bookingHead; if (currentBooking == NULL) { printf("\nNo confirmed bookings.\n"); return; } printf("\n========== CONFIRMED BOOKINGS ==========\n"); while (currentBooking != NULL) { printf("Booking ID: %d | Passenger: %s\n", currentBooking->bookingID, currentBooking->passengerName); printf("Age: %d | Flight: %d\n\n", currentBooking->age, currentBooking->flightNo); currentBooking = currentBooking->next; } } /* Show a booking; passengers must also enter the booking name. */ void viewBooking(int passengerMode) { int bookingID; int flightIndex; char passengerName[NAME_LEN]; Booking *booking; bookingID = readInt("\nEnter booking ID: "); booking = findBooking(bookingID); if (booking == NULL) { printf("Booking not found.\n"); return; } if (passengerMode) { readLine("Enter passenger name used for booking: ", passengerName, sizeof(passengerName)); if (strcmp(passengerName, booking->passengerName) != 0) { printf("Passenger name does not match.\n"); return; } } flightIndex = findFlightIndex(booking->flightNo); printf("\n========== SKYLINE AIRLINES ==========\n"); printf(" E-TICKET\n"); printf("--------------------------------------\n"); printf("Booking ID : %d\n", booking->bookingID); printf("Passenger : %s\n", booking->passengerName); printf("Age : %d\n", booking->age); printf("Flight : %d\n", booking->flightNo); if (flightIndex != -1) { printf("Route : %s to %s\n", flights[flightIndex].source, flights[flightIndex].destination); printf("Date : %s\n", flights[flightIndex].date); printf("Time : %s\n", flights[flightIndex].time); printf("Fare : Rs. %.2f\n", flights[flightIndex].fare); } printf("======================================\n"); } /* Add a passenger to the back of the waiting queue. */ void enqueueWaiting(const char *name, int age, int flightNo) { QueueNode *newPassenger; newPassenger = (QueueNode *)malloc(sizeof(QueueNode)); if (newPassenger == NULL) { printf("Could not add passenger to waiting list.\n"); return; } snprintf(newPassenger->name, sizeof(newPassenger->name), "%s", name); newPassenger->age = age; newPassenger->flightNo = flightNo; newPassenger->next = NULL; /* The first item is both the front and rear of an empty queue. */ if (waitingList.rear == NULL) { waitingList.front = newPassenger; waitingList.rear = newPassenger; } else { waitingList.rear->next = newPassenger; waitingList.rear = newPassenger; } printf("\nThis flight is full.\n"); printf("You have been added to the waiting list.\n"); } /* Print passengers from the front to the back of the queue. **Result:** No value is returned.

bookTicket()

```
void bookTicket(void)
```

In simple words: Find and return a booking by its ID. */ Booking *findBooking(int bookingID) { Booking *currentBooking = bookingHead; while (currentBooking != NULL) { if (currentBooking->bookingID == bookingID) { return currentBooking; } currentBooking = currentBooking->next; } return NULL; } /* Make a booking node and attach it to the end of the booking list. */ int appendBooking(int bookingID, const char *name, int age, int flightNo) { Booking *newBooking; Booking *lastBooking; newBooking = (Booking *)malloc(sizeof(Booking)); if (newBooking == NULL) { printf("Memory allocation failed.\n"); return 0; } newBooking->bookingID = bookingID; newBooking->age = age; newBooking->flightNo = flightNo; snprintf(newBooking->passengerName, sizeof(newBooking->passengerName), "%s", name); newBooking->next = NULL; /* Add the new booking at the end of the singly linked list. */ if (bookingHead == NULL) { bookingHead = newBooking; } else { lastBooking = bookingHead; while (lastBooking->next != NULL) { lastBooking = lastBooking->next; } lastBooking->next = newBooking; } return 1; } /* Print each booking in the linked list. */ void displayAllBookings(void) { Booking *currentBooking = bookingHead; if (currentBooking == NULL) { printf("\nNo confirmed bookings.\n"); return; } printf("\n========== CONFIRMED BOOKINGS ==========\n"); while (currentBooking != NULL) { printf("Booking ID: %d | Passenger: %s\n", currentBooking->bookingID, currentBooking->passengerName); printf("Age: %d | Flight: %d\n\n", currentBooking->age, currentBooking->flightNo); currentBooking = currentBooking->next; } } /* Show a booking; passengers must also enter the booking name. */ void viewBooking(int passengerMode) { int bookingID; int flightIndex; char passengerName[NAME_LEN]; Booking *booking; bookingID = readInt("\nEnter booking ID: "); booking = findBooking(bookingID); if (booking == NULL) { printf("Booking not found.\n"); return; } if (passengerMode) { readLine("Enter passenger name used for booking: ", passengerName, sizeof(passengerName)); if (strcmp(passengerName, booking->passengerName) != 0) { printf("Passenger name does not match.\n"); return; } } flightIndex = findFlightIndex(booking->flightNo); printf("\n========== SKYLINE AIRLINES ==========\n"); printf(" E-TICKET\n"); printf("--------------------------------------\n"); printf("Booking ID : %d\n", booking->bookingID); printf("Passenger : %s\n", booking->passengerName); printf("Age : %d\n", booking->age); printf("Flight : %d\n", booking->flightNo); if (flightIndex != -1) { printf("Route : %s to %s\n", flights[flightIndex].source, flights[flightIndex].destination); printf("Date : %s\n", flights[flightIndex].date); printf("Time : %s\n", flights[flightIndex].time); printf("Fare : Rs. %.2f\n", flights[flightIndex].fare); } printf("======================================\n"); } /* Add a passenger to the back of the waiting queue. */ void enqueueWaiting(const char *name, int age, int flightNo) { QueueNode *newPassenger; newPassenger = (QueueNode *)malloc(sizeof(QueueNode)); if (newPassenger == NULL) { printf("Could not add passenger to waiting list.\n"); return; } snprintf(newPassenger->name, sizeof(newPassenger->name), "%s", name); newPassenger->age = age; newPassenger->flightNo = flightNo; newPassenger->next = NULL; /* The first item is both the front and rear of an empty queue. */ if (waitingList.rear == NULL) { waitingList.front = newPassenger; waitingList.rear = newPassenger; } else { waitingList.rear->next = newPassenger; waitingList.rear = newPassenger; } printf("\nThis flight is full.\n"); printf("You have been added to the waiting list.\n"); } /* Print passengers from the front to the back of the queue. */ void displayWaitingList(void) { QueueNode *currentPassenger = waitingList.front; int position = 1; if (currentPassenger == NULL) { printf("\nWaiting list is empty.\n"); return; } printf("\n========== WAITING LIST ==========\n"); while (currentPassenger != NULL) { printf("%d. %s | Age: %d | Flight: %d\n", position, currentPassenger->name, currentPassenger->age, currentPassenger->flightNo); position++; currentPassenger = currentPassenger->next; } } /* Book a seat, or add the passenger to the queue when the flight is full. **Result:** No value is returned.

promoteWaitingPassenger()

```
void promoteWaitingPassenger(int flightNo)
```

In simple words: Find and return a booking by its ID. */ Booking *findBooking(int bookingID) { Booking *currentBooking = bookingHead; while (currentBooking != NULL) { if (currentBooking->bookingID == bookingID) { return currentBooking; } currentBooking = currentBooking->next; } return NULL; } /* Make a booking node and attach it to the end of the booking list. */ int appendBooking(int bookingID, const char *name, int age, int flightNo) { Booking *newBooking; Booking *lastBooking; newBooking = (Booking *)malloc(sizeof(Booking)); if (newBooking == NULL) { printf("Memory allocation failed.\n"); return 0; } newBooking->bookingID = bookingID; newBooking->age = age; newBooking->flightNo = flightNo; snprintf(newBooking->passengerName, sizeof(newBooking->passengerName), "%s", name); newBooking->next = NULL; /* Add the new booking at the end of the singly linked list. */ if (bookingHead == NULL) { bookingHead = newBooking; } else { lastBooking = bookingHead; while (lastBooking->next != NULL) { lastBooking = lastBooking->next; } lastBooking->next = newBooking; } return 1; } /* Print each booking in the linked list. */ void displayAllBookings(void) { Booking *currentBooking = bookingHead; if (currentBooking == NULL) { printf("\nNo confirmed bookings.\n"); return; } printf("\n========== CONFIRMED BOOKINGS ==========\n"); while (currentBooking != NULL) { printf("Booking ID: %d | Passenger: %s\n", currentBooking->bookingID, currentBooking->passengerName); printf("Age: %d | Flight: %d\n\n", currentBooking->age, currentBooking->flightNo); currentBooking = currentBooking->next; } } /* Show a booking; passengers must also enter the booking name. */ void viewBooking(int passengerMode) { int bookingID; int flightIndex; char passengerName[NAME_LEN]; Booking *booking; bookingID = readInt("\nEnter booking ID: "); booking = findBooking(bookingID); if (booking == NULL) { printf("Booking not found.\n"); return; } if (passengerMode) { readLine("Enter passenger name used for booking: ", passengerName, sizeof(passengerName)); if (strcmp(passengerName, booking->passengerName) != 0) { printf("Passenger name does not match.\n"); return; } } flightIndex = findFlightIndex(booking->flightNo); printf("\n========== SKYLINE AIRLINES ==========\n"); printf(" E-TICKET\n"); printf("--\n"); printf("Booking ID : %d\n", booking->bookingID); printf("Passenger : %s\n", booking->passengerName); printf("Age : %d\n", booking->age); printf("Flight : %d\n", booking->flightNo); if (flightIndex != -1) { printf("Route : %s to %s\n", flights[flightIndex].source, flights[flightIndex].destination); printf("Date : %s\n", flights[flightIndex].date); printf("Time : %s\n", flights[flightIndex].time); printf("Fare : Rs. %.2f\n", flights[flightIndex].fare); } printf("==\n"); } /* Add a passenger to the back of the waiting queue. */ void enqueueWaiting(const char *name, int age, int flightNo) { QueueNode *newPassenger; newPassenger = (QueueNode *)malloc(sizeof(QueueNode)); if (newPassenger == NULL) { printf("Could not add passenger to waiting list.\n"); return; } snprintf(newPassenger->name, sizeof(newPassenger->name), "%s", name); newPassenger->age = age; newPassenger->flightNo = flightNo; newPassenger->next = NULL; /* The first item is both the front and rear of an empty queue. */ if (waitingList.rear == NULL) { waitingList.front = newPassenger; waitingList.rear = newPassenger; } else { waitingList.rear->next = newPassenger; waitingList.rear = newPassenger; } printf("\nThis flight is full.\n"); printf("You have been added to the waiting list.\n"); } /* Print passengers from the front to the back of the queue. */ void displayWaitingList(void) { QueueNode *currentPassenger = waitingList.front; int position = 1; if (currentPassenger == NULL) { printf("\nWaiting list is empty.\n"); return; } printf("\n========== WAITING LIST ==========\n"); while (currentPassenger != NULL) { printf("%d. %s | Age: %d | Flight: %d\n", position, currentPassenger->name, currentPassenger->age, currentPassenger->flightNo); position++; currentPassenger = currentPassenger->next; } } /* Book a seat, or add the passenger to the queue when the flight is full. */ void bookTicket(void) { char name[NAME_LEN]; int passengerAge; int flightNumber; int flightIndex; int bookingID; readLine("\nPassenger full name: ", name, sizeof(name)); if (name[0] == '\0') { printf("Name cannot be empty.\n"); return; } passengerAge = readInt("Passenger age: "); if (passengerAge < 1 || passengerAge > 120) { printf("Enter an age between 1 and 120.\n"); return; } displayFlights(); flightNumber = readInt("\nEnter flight number to book: "); flightIndex = findFlightIndex(flightNumber); if (flightIndex == -1) { printf("Invalid flight number.\n"); return; } if (flights[flightIndex].availableSeats == 0) { enqueueWaiting(name, passengerAge, flightNumber); return; } bookingID = nextBookingID++; if (!appendBooking(bookingID, name, passengerAge, flightNumber)) { nextBookingID--; return; } flights[flightIndex].availableSeats--; printf("\n"); printf("==\n"); printf(" SKYLINE AIRLINES\n"); printf(" E-TICKET\n"); printf("==\n"); printf("Booking ID : %d\n", bookingID); printf("Passenger : %s\n", name); printf("Age : %d\n", passengerAge); printf("Flight : %d\n", flightNumber); printf("Route : %s to %s\n", flights[flightIndex].source, flights[flightIndex].destination); printf("Travel date : %s\n", flights[flightIndex].date); printf("Departure time : %s\n", flights[flightIndex].time); printf("Ticket fare : Rs. %.2f\n", flights[flightIndex].fare); printf("Seats remaining : %d\n", flights[flightIndex].availableSeats); printf("==\n"); printf("Keep your booking ID for future reference.\n"); } /* Move the first waiting passenger for this flight into confirmed bookings. **Result:** No value is returned.

cancelTicket()

```
void cancelTicket(int passengerMode)
```

In simple words: Find and return a booking by its ID. */ Booking *findBooking(int bookingID) { Booking *currentBooking = bookingHead; while (currentBooking != NULL) { if (currentBooking->bookingID == bookingID) { return currentBooking; } currentBooking = currentBooking->next; } return NULL; } /* Make a booking node and attach it to the end of the booking list. */ int appendBooking(int bookingID, const char *name, int age, int flightNo) { Booking *newBooking; Booking *lastBooking; newBooking = (Booking *)malloc(sizeof(Booking)); if (newBooking == NULL) { printf("Memory allocation failed.\n"); return 0; } newBooking->bookingID = bookingID; newBooking->age = age; newBooking->flightNo = flightNo; snprintf(newBooking->passengerName, sizeof(newBooking->passengerName), "%s", name); newBooking->next = NULL; /* Add the new booking at the end of the singly linked list. */ if (bookingHead == NULL) { bookingHead = newBooking; } else { lastBooking = bookingHead; while (lastBooking->next != NULL) { lastBooking = lastBooking->next; } lastBooking->next = newBooking; } return 1; } /* Print each booking in the linked list. */ void displayAllBookings(void) { Booking *currentBooking = bookingHead; if (currentBooking == NULL) { printf("\nNo confirmed bookings.\n"); return; } printf("\n========== CONFIRMED BOOKINGS ==========\n"); while (currentBooking != NULL) { printf("Booking ID: %d | Passenger: %s\n", currentBooking->bookingID, currentBooking->passengerName); printf("Age: %d | Flight: %d\n\n", currentBooking->age, currentBooking->flightNo); currentBooking = currentBooking->next; } } /* Show a booking; passengers must also enter the booking name. */ void viewBooking(int passengerMode) { int bookingID; int flightIndex; char passengerName[NAME_LEN]; Booking *booking; bookingID = readInt("\nEnter booking ID: "); booking = findBooking(bookingID); if (booking == NULL) { printf("Booking not found.\n"); return; } if (passengerMode) { readLine("Enter passenger name used for booking: ", passengerName, sizeof(passengerName)); if (strcmp(passengerName, booking->passengerName) != 0) { printf("Passenger name does not match.\n"); return; } } flightIndex = findFlightIndex(booking->flightNo); printf("\n========== SKYLINE AIRLINES ==========\n"); printf(" E-TICKET\n"); printf("--\n"); printf("Booking ID : %d\n", booking->bookingID); printf("Passenger : %s\n", booking->passengerName); printf("Age : %d\n", booking->age); printf("Flight : %d\n", booking->flightNo); if (flightIndex != -1) { printf("Route : %s to %s\n", flights[flightIndex].source, flights[flightIndex].destination); printf("Date : %s\n", flights[flightIndex].date); printf("Time : %s\n", flights[flightIndex].time); printf("Fare : Rs. %.2f\n", flights[flightIndex].fare); } printf("==\n"); } /* Add a passenger to the back of the waiting queue. */ void enqueueWaiting(const char *name, int age, int flightNo) { QueueNode *newPassenger; newPassenger = (QueueNode *)malloc(sizeof(QueueNode)); if (newPassenger == NULL) { printf("Could not add passenger to waiting list.\n"); return; } snprintf(newPassenger->name, sizeof(newPassenger->name), "%s", name); newPassenger->age = age; newPassenger->flightNo = flightNo; newPassenger->next = NULL; /* The first item is both the front and rear of an empty queue. */ if (waitingList.rear == NULL) { waitingList.front = newPassenger; waitingList.rear = newPassenger; } else { waitingList.rear->next = newPassenger; waitingList.rear = newPassenger; } printf("\nThis flight is full.\n"); printf("You have been added to the waiting list.\n"); } /* Print passengers from the front to the back of the queue. */ void displayWaitingList(void) { QueueNode *currentPassenger = waitingList.front; int position = 1; if (currentPassenger == NULL) { printf("\nWaiting list is empty.\n"); return; } printf("\n========== WAITING LIST ==========\n"); while (currentPassenger != NULL) { printf("%d. %s | Age: %d | Flight: %d\n", position, currentPassenger->name, currentPassenger->age, currentPassenger->flightNo); position++; currentPassenger = currentPassenger->next; } } /* Book a seat, or add the passenger to the queue when the flight is full. */ void bookTicket(void) { char name[NAME_LEN]; int passengerAge; int flightNumber; int flightIndex; int bookingID; readLine("\nPassenger full name: ", name, sizeof(name)); if (name[0] == '\0') { printf("Name cannot be empty.\n"); return; } passengerAge = readInt("Passenger age: "); if (passengerAge < 1 || passengerAge > 120) { printf("Enter an age between 1 and 120.\n"); return; } displayFlights(); flightNumber = readInt("\nEnter flight number to book: "); flightIndex = findFlightIndex(flightNumber); if (flightIndex == -1) { printf("Invalid flight number.\n"); return; } if (flights[flightIndex].availableSeats == 0) { enqueueWaiting(name, passengerAge, flightNumber); return; } bookingID = nextBookingID++; if (!appendBooking(bookingID, name, passengerAge, flightNumber)) { nextBookingID--; return; } flights[flightIndex].availableSeats--; printf("\n"); printf("==\n"); printf(" SKYLINE AIRLINES\n"); printf(" E-TICKET\n"); printf("==\n"); printf("Booking ID : %d\n", bookingID); printf("Passenger : %s\n", name); printf("Age : %d\n", passengerAge); printf("Flight : %d\n", flightNumber); printf("Route : %s to %s\n", flights[flightIndex].source, flights[flightIndex].destination); printf("Travel date : %s\n", flights[flightIndex].date); printf("Departure time : %s\n", flights[flightIndex].time); printf("Ticket fare : Rs. %.2f\n", flights[flightIndex].fare); printf("Seats remaining : %d\n", flights[flightIndex].availableSeats); printf("==\n"); printf("Keep your booking ID for future reference.\n"); } /* Move the first waiting passenger for this flight into confirmed bookings. */ void promoteWaitingPassenger(int flightNo) { QueueNode *currentPassenger = waitingList.front; QueueNode *previousPassenger = NULL; int flightIndex = findFlightIndex(flightNo); int bookingID; if (flightIndex == -1 || flights[flightIndex].availableSeats <= 0) { return; } /* Find the earliest queued passenger for this flight. */ while (currentPassenger != NULL && currentPassenger->flightNo != flightNo) { previousPassenger = currentPassenger; currentPassenger = currentPassenger->next; } if (currentPassenger == NULL) { return; } /* Confirm the booking before removing the passenger from the queue. */ bookingID = nextBookingID; if (!appendBooking(bookingID, currentPassenger->name, currentPassenger->age, flightNo)) { return; } /* Unlink the passenger while keeping the queue's front and rear correct. */ if (previousPassenger == NULL) { waitingList.front = currentPassenger->next; } else { previousPassenger->next = currentPassenger->next; } if (waitingList.rear == currentPassenger) { waitingList.rear = previousPassenger; } nextBookingID++; flights[flightIndex].availableSeats--; printf("\nWaiting-list passenger confirmed!\n"); printf("Passenger: %s\n", currentPassenger->name); printf("New booking ID: %d\n", bookingID); free(currentPassenger); } /* Cancel a booking and offer its seat to the next waiting passenger. **Result:** No value is returned.

storage.c

Reads and writes saved flights and confirmed bookings.

saveFlights()

```
void saveFlights(void)
```

In simple words: Write all flights to the pipe-separated flights.txt file. **Result:** No value is returned.

loadFlights()

```
int loadFlights(void)
```

In simple words: Write all flights to the pipe-separated flights.txt file. */ void saveFlights(void) { FILE *file = fopen("flights.txt", "w"); int flightIndex; if (file == NULL) { printf("Could not save flights.txt\n"); return; } /* Save each array item as one pipe-separated text record. */ for (flightIndex = 0; flightIndex < flightCount; flightIndex++) { fprintf(file, "%d|%s|%s|%s|%s|%d|%d|%.2f\n", flights[flightIndex].flightNo, flights[flightIndex].source, flights[flightIndex].destination, flights[flightIndex].date, flights[flightIndex].time, flights[flightIndex].totalSeats, flights[flightIndex].availableSeats, flights[flightIndex].fare); } fclose(file); } /* Load saved flights; return 1 when at least one flight was read. **Result:** Returns 1 if at least one flight was read; otherwise returns 0.

saveBookings()

```
void saveBookings(void)
```

In simple words: Write all flights to the pipe-separated flights.txt file. */ void saveFlights(void) { FILE *file = fopen("flights.txt", "w"); int flightIndex; if (file == NULL) { printf("Could not save flights.txt\n"); return; } /* Save each array item as one pipe-separated text record. */ for (flightIndex = 0; flightIndex < flightCount; flightIndex++) { fprintf(file, "%d|%s|%s|%s|%s|%d|%d|%.2f\n", flights[flightIndex].flightNo, flights[flightIndex].source, flights[flightIndex].destination, flights[flightIndex].date, flights[flightIndex].time, flights[flightIndex].totalSeats, flights[flightIndex].availableSeats, flights[flightIndex].fare); } fclose(file); } /* Load saved flights; return 1 when at least one flight was read. */ int loadFlights(void) { FILE *file = fopen("flights.txt", "r"); Flight flight; if (file == NULL) { return 0; } /* Each %[^|] reads text up to the next pipe separator. */ while (flightCount < MAX_FLIGHTS && fscanf(file, "%d|%29[^|]|%29[^|]|%14[^|]|%9[^|]|%d|%d|%f\n", &flight.flightNo, flight.source, flight.destination, flight.date, flight.time, &flight.totalSeats, &flight.availableSeats, &flight.fare) == 8) { flights[flightCount] = flight; flightCount++; } fclose(file); return flightCount > 0; } /* Write each confirmed booking to bookings.txt. **Result:** No value is returned.

loadBookings()

```
void loadBookings(void)
```

In simple words: Write all flights to the pipe-separated flights.txt file. */ void saveFlights(void) { FILE *file = fopen("flights.txt", "w"); int flightIndex; if (file == NULL) { printf("Could not save flights.txt\n"); return; } /* Save each array item as one pipe-separated text record. */ for (flightIndex = 0; flightIndex < flightCount; flightIndex++) { fprintf(file, "%d|%s|%s|%s|%s|%d|%d|%.2f\n", flights[flightIndex].flightNo, flights[flightIndex].source, flights[flightIndex].destination, flights[flightIndex].date, flights[flightIndex].time, flights[flightIndex].totalSeats, flights[flightIndex].availableSeats, flights[flightIndex].fare); } fclose(file); } /* Load saved flights; return 1 when at least one flight was read. */ int loadFlights(void) { FILE *file = fopen("flights.txt", "r"); Flight flight; if (file == NULL) { return 0; } /* Each %[^|] reads text up to the next pipe separator. */ while (flightCount < MAX_FLIGHTS && fscanf(file, "%d|%29[^|]|%29[^|]|%14[^|]|%9[^|]|%d|%d|%f\n", &flight.flightNo, flight.source, flight.destination, flight.date, flight.time, &flight.totalSeats, &flight.availableSeats, &flight.fare) == 8) { flights[flightCount] = flight; flightCount++; } fclose(file); return flightCount > 0; } /* Write each confirmed booking to bookings.txt. */ void saveBookings(void) { FILE *file = fopen("bookings.txt", "w"); Booking *currentBooking = bookingHead; if (file == NULL) { printf("Could not save bookings.txt\n"); return; } while (currentBooking != NULL) { fprintf(file, "%d|%s|%d|%d\n", currentBooking->bookingID, currentBooking->passengerName, currentBooking->age, currentBooking->flightNo); currentBooking = currentBooking->next; } fclose(file); } /* Load bookings into the linked list and find the next unused booking ID. **Result:** No value is returned.

saveAll()

```
void saveAll(void)
```

In simple words: Write all flights to the pipe-separated flights.txt file. */ void saveFlights(void) { FILE *file = fopen("flights.txt", "w"); int flightIndex; if (file == NULL) { printf("Could not save flights.txt\n"); return; } /* Save each array item as one pipe-separated text record. */ for (flightIndex = 0; flightIndex < flightCount; flightIndex++) { fprintf(file, "%d|%s|%s|%s|%s|%d|%d|%.2f\n", flights[flightIndex].flightNo, flights[flightIndex].source, flights[flightIndex].destination, flights[flightIndex].date, flights[flightIndex].time, flights[flightIndex].totalSeats, flights[flightIndex].availableSeats, flights[flightIndex].fare); } fclose(file); } /* Load saved flights; return 1 when at least one flight was read. */ int loadFlights(void) { FILE *file = fopen("flights.txt", "r"); Flight flight; if (file == NULL) { return 0; } /* Each %[^|] reads text up to the next pipe separator. */ while (flightCount < MAX_FLIGHTS && fscanf(file, "%d|%29[^|]|%29[^|]|%14[^|]|%9[^|]|%d|%d|%f\n", &flight.flightNo, flight.source, flight.destination, flight.date, flight.time, &flight.totalSeats, &flight.availableSeats, &flight.fare) == 8) { flights[flightCount] = flight; flightCount++; } fclose(file); return flightCount > 0; } /* Write each confirmed booking to bookings.txt. */ void saveBookings(void) { FILE *file = fopen("bookings.txt", "w"); Booking *currentBooking = bookingHead; if (file == NULL) { printf("Could not save bookings.txt\n"); return; } while (currentBooking != NULL) { fprintf(file, "%d|%s|%d|%d\n", currentBooking->bookingID, currentBooking->passengerName, currentBooking->age, currentBooking->flightNo); currentBooking = currentBooking->next; } fclose(file); } /* Load bookings into the linked list and find the next unused booking ID. */ void loadBookings(void) { FILE *file = fopen("bookings.txt", "r"); int bookingID; int passengerAge; int flightNo; char passengerName[NAME_LEN]; if (file == NULL) { return; } /* Read saved records and rebuild the in-memory booking linked list. */ while (fscanf(file, "%d|%49[^|]|%d|%d\n", &bookingID, passengerName, &passengerAge, &flightNo) == 4) { if (findFlightIndex(flightNo) != -1 && appendBooking(bookingID, passengerName, passengerAge, flightNo)) { if (bookingID >= nextBookingID) { nextBookingID = bookingID + 1; } } } fclose(file); } /* Save both kinds of permanent data. **Result:** No value is returned.

menus.c

Shows passenger/admin menus and calls the selected feature.

passengerMenu()

```
void passengerMenu(void)
```

In simple words: Show the passenger menu and run the selected action. **Result:** No value is returned.

ownerLogin()

```
int ownerLogin(void)
```

In simple words: Show the passenger menu and run the selected action. */ void passengerMenu(void) { int menuChoice; do { printf("\n--- Passenger Portal ---\n"); printf("1. View available flights\n"); printf("2. Search flights by route\n"); printf("3. Search flight by number\n"); printf("4. Book a ticket\n"); printf("5. View my booking\n"); printf("6. Cancel my booking\n"); printf("0. Back to main menu\n"); menuChoice = readInt("Enter your choice: "); switch (menuChoice) { case 1: displayFlights(); break; case 2: searchFlightsByRoute(); break; case 3: searchFlightByNumber(); break; case 4: bookTicket(); saveAll(); break; case 5: viewBooking(1); break; case 6: cancelTicket(1); saveAll(); break; case 0: printf("Returning to main menu...\n"); break; default: printf("Invalid choice. Try again.\n"); } if (menuChoice != 0) { pauseScreen(); } } while (menuChoice != 0); } /* Ask for the admin ID; allow up to three attempts. **Result:** Returns 1 for a correct ID; otherwise returns 0.

ownerMenu()

```
void ownerMenu(void)
```

In simple words: Show the passenger menu and run the selected action. */ void passengerMenu(void) { int menuChoice; do { printf("\n--- Passenger Portal ---\n"); printf("1. View available flights\n"); printf("2. Search flights by route\n"); printf("3. Search flight by number\n"); printf("4. Book a ticket\n"); printf("5. View my booking\n"); printf("6. Cancel my booking\n"); printf("0. Back to main menu\n"); menuChoice = readInt("Enter your choice: "); switch (menuChoice) { case 1: displayFlights(); break; case 2: searchFlightsByRoute(); break; case 3: searchFlightByNumber(); break; case 4: bookTicket(); saveAll(); break; case 5: viewBooking(1); break; case 6: cancelTicket(1); saveAll(); break; case 0: printf("Returning to main menu...\n"); break; default: printf("Invalid choice. Try again.\n"); } if (menuChoice != 0) { pauseScreen(); } } while (menuChoice != 0); } /* Ask for the admin ID; allow up to three attempts. */ int ownerLogin(void) { char enteredID[60]; int loginAttempt; printf("\n--- Owner / Admin Login ---\n"); for (loginAttempt = 1; loginAttempt <= 3; loginAttempt++) { readLine("Enter owner ID: ", enteredID, sizeof(enteredID)); if (strcmp(enteredID, OWNER_ID) == 0) { printf("\nAccess granted. Welcome, Administrator!\n"); return 1; } printf("Incorrect ID. Attempts remaining: %d\n", 3 - loginAttempt); } printf("Access denied. Returning to main menu.\n"); return 0; } /* Check the admin ID, then show the admin menu. **Result:** No value is returned.

5. Other files and saved data

.gitignore

Keeps generated build output, local data files, and Qt Creator settings out of Git.

README.md

Introduces the project and explains its features, structures, and how to run it.

run.bat

Changes to the project folder, creates the build folder if needed, compiles all C files with warnings, stops after a compiler error, and starts the app.

flights.txt

Each line stores a flight number, route, schedule, total seats, available seats, and fare. Fields are separated by |.

bookings.txt

Each line stores a booking ID, passenger name and age, and flight number. Fields are separated by |.

build/skyline-airlines.exe

Generated by run.bat. This is the compiled program, not a source file.

```
Flight:  101|Delhi|Mumbai|2026-11-10|09:30|20|20|4000.00
Booking: 1001|Asha Singh|30|101
```

6. Short project explanation

“Skyline Airlines is a C99 console application. Flights are stored in an array. Confirmed bookings use a linked list, waiting passengers use a FIFO queue, and an admin option searches flight numbers in a binary search tree. The program saves flights and confirmed bookings in text files. Separate files organize input, flight features, booking features, storage, and menus.”

Important limits

• There can be at most 50 flights.

• The admin ID is hard-coded and is not secure authentication.

• The waiting queue is cleared when the program exits.

• The binary search tree is not self-balancing.

• A pipe character cannot safely appear inside a saved name or city.